

# Lab 01 — Containers and architecture: Docker, Podman and the Linux kernel

---

Theory — what a container really is, who does what when you type `podman run`, and why "Docker" now means both a tool and a way of working.

## Objectives

---

- Be able to say what a container is **without** using the words "lightweight virtual machine".
  - Name the two Linux kernel mechanisms that make containers possible.
  - Tell the actors apart: client, engine (daemon or not), image, registry — for Docker **and** for Podman.
  - Tell an **image** from a **container** — the most expensive confusion for a beginner.
  - Read any `docker` / `podman` command and guess its structure.
- 

## 1. A story of "it works on my machine"

---

An application never runs on its own. A Spring Boot API needs a JRE in a precise version, environment variables, a certificate, a time zone. That whole set is called the **runtime environment**, and for twenty years it was installed by hand on servers. The result: the developer's workstation and production were never identical, and two applications on the same server fought over the same library in two versions.

The first answer was the **virtual machine**: a complete simulated computer, with its own operating system, per application. It works — at the price of several GB of disk, reserved RAM and a boot measured in minutes, to run *one* process.

### → HISTORY

In March 2013, Solomon Hykes presents Docker in five minutes at the PyCon conference. Nothing is technically new: *namespaces* and *cgroups* have been in Linux since 2008, LXC already uses them. What Docker invents is the **packaging**: an image you build, publish and run with one command. In 2015 Docker hands the image format and the runtime over to the **OCI** (*Open Container Initiative*): since then an image is a standard that any tool can run. Podman (Red Hat, 2018) was born from that opening.

The container is the second answer: **isolate the process, not the machine**.

## 2. What a container is

---

### REMEMBER

A container is an **ordinary process** on your Linux machine, to which the kernel lies about what it can see and what it can consume.

There is no operating system inside a container. No emulation. If you start an `nginx` container, a real `nginx` process exists on your host, visible in `ps aux`, run by the **same Linux kernel** as everything else. What changes is what that process perceives of the world. Two kernel mechanisms do that work.

#### → LINUX

The **kernel** is the part of the system that talks to the hardware and arbitrates between programs: it creates processes, gives them memory, CPU time, access to files and to the network. Everything "above" it — `bash`, `ls`, `java`, `nginx` — is called **userland**. A program never touches the hardware directly: it asks the kernel through **system calls** (`open`, `read`, `fork` ...). That boundary is what containers exploit.

**Namespaces — isolating the view.** The kernel gives a process a partial, private view of certain resources:

| Namespace         | What it isolates            | Visible consequence                                                                                                    |
|-------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>pid</code>  | Process identifiers         | Inside the container your application is PID 1 and sees nothing else                                                   |
| <code>net</code>  | Interfaces, ports, routes   | The container has its own port 8080, distinct from the host's                                                          |
| <code>mnt</code>  | Mount points                | The container sees its own <code>/</code>                                                                              |
| <code>uts</code>  | The hostname                | <code>hostname</code> returns the container ID                                                                         |
| <code>ipc</code>  | Inter-process communication | No shared memory with the neighbours                                                                                   |
| <code>user</code> | UIDs/GIDs                   | A <code>root</code> inside the container may be a plain user on the host — <b>this is the heart of rootless Podman</b> |

**Cgroups (control groups) — limiting resources.** Namespaces hide, cgroups cap: "this group of processes gets no more than 512 MB of RAM and 1.5 cores". That is what stops a runaway container from taking the server down with it.

### Container or VM?

|                   | Virtual machine                         | Container                              |
|-------------------|-----------------------------------------|----------------------------------------|
| Isolates          | A whole computer (virtualised hardware) | One or more processes                  |
| Kernel            | Its own                                 | <b>The host's, shared</b>              |
| Start-up          | Seconds to minutes                      | Milliseconds                           |
| Typical weight    | Several GB                              | A few dozen MB                         |
| Security boundary | Strong (hypervisor)                     | Weaker (a single kernel to compromise) |

#### → WINDOWS / WSL

"The container shares the host kernel" has a consequence: a Linux container runs **only** on a Linux kernel. On Windows, **WSL 2** (*Windows Subsystem for Linux*) provides it: a very light VM, managed by Hyper-V, that boots in a second, shares RAM with Windows and runs a real Linux kernel compiled by Microsoft. Your Podman runs *inside* that Ubuntu distribution; your containers are processes of that VM, not of Windows. Docker Desktop and Podman Desktop do the same thing behind the scenes: they create their own WSL distribution.

## 3. Image and container

This is the fundamental distinction of the whole course.

An **image** is a **read-only** template: a frozen file system (the JRE, your JAR, the libraries) plus metadata (which command to launch, which variables, which user, which port). An image does not execute, consumes no CPU, does not "run". It is inert and **immutable**.

A **container** is a *running instance* of an image: the image, plus a thin writable layer private to that instance, plus a live process.

#### → JAVA

The most accurate analogy comes from object-oriented programming: the image is the **class**, the container is the **object** ( `new` ). You instantiate twenty containers from the same image; they share the same read-only content and each has its own private state. And like an object, a container is destroyed without the class moving.

#### REMEMBER

Everything your application writes inside a container goes into that writable layer, which is **destroyed with the container**. That is intended: a container is disposable. Persistence is the topic of lab 06.

An image is made of stacked **layers**, one per build step; ten images built on the same `eclipse-temurin:21-jre` store that base only once (lab 02).

## 4. The architecture: who does what

Here Docker and Podman diverge — and that is Podman's reason to exist.

|        |                                 |                 |                                       |   |                         |   |                   |
|--------|---------------------------------|-----------------|---------------------------------------|---|-------------------------|---|-------------------|
| DOCKER | <code>docker</code> (client)    | → HTTP/socket → | <code>dockerd</code> (daemon, root)   | → | <code>containerd</code> | → | <code>runc</code> |
| PODMAN | <code>podman</code> (your user) | → fork/exec →   | <code>conmon</code>                   | → | <code>crun</code>       |   | (no daemon)       |
|        |                                 | both → pull →   | Registry (Docker Hub, Harbor, ECR...) |   |                         |   |                   |

**Docker** is a **client / server** architecture. The `docker` binary does almost nothing: it turns your command into an HTTP request to `dockerd`, a permanent **daemon**, running as **root**, that does all the work (build, create, store) and listens on a Unix *socket*, `/var/run/docker.sock`.

**Podman** has **no daemon**. Each `podman` command is an ordinary program that does the work itself and then exits; the container survives thanks to a tiny supervisor, `conmon`, that stays attached to it. Above all, Podman runs **rootless** by default: it is *your* user that launches the container, without privilege. The `root` you will see inside the container is an illusion of the `user` namespace: on the host side, it is you.

#### PODMAN

Why a second tool? Two complaints operations teams made about Docker: **a permanent root daemon** (a single point of failure, and "whoever can talk to the socket is root") and **a licence** (Docker Desktop has been paid-for in companies since 2021). Podman answers both: no daemon, rootless by default, free. And it made a decisive choice: its CLI is **identical** to Docker's. `alias docker=podman` is enough in 95% of cases; images, Dockerfiles and registries are the same, because all of that is OCI. So you learn *Docker* — the vocabulary of the workplace — with Podman as the engine.

The two share the rest: the **registry**, the remote image store (Docker Hub by default; in companies Harbor, Nexus, GitLab Registry, ECR, ACR), and the low-level **runtime** ( `runc` or `crun` ), which actually asks the kernel to create the namespaces. Its name shows up in error messages.

Two practical consequences to understand right now:

1. **With Docker, the work happens on the daemon side.** A path mounted with `-v /data:/data` is resolved on the *daemon's* disk, not the client's — invisible locally, the source of half the surprises against a remote daemon. With Podman, client and engine are the same process.
2. **Access to the Docker daemon = root access on the host.** Whoever can write to `/var/run/docker.sock` can start a privileged container and take over the machine.

#### → SECURITY

Membership of the `docker` group is equivalent to `sudo` without a password or an audit trail. Rootless Podman is the structural answer: a compromised container has only *your* rights.

## 5. Anatomy of a command

The CLI follows a regular grammar, the same for both tools:

```
podman [object] [action] [options] [target] [arguments]
```

```
podman container run -d --name api -p 8080:8080 docker.io/library/eclipse-temurin:21-jre java -
version
#      ^object^  ^action^  ^options^  ^image^  ^
command ^
```

The main objects are `image`, `container`, `volume`, `network`, `system` (and `pod`, specific to Podman). For the most frequent operations there are **historical shortcuts** where the object is implicit — those are the ones you read everywhere:

| Full form                                                       | Common shortcut                                  |
|-----------------------------------------------------------------|--------------------------------------------------|
| <code>podman container run</code>                               | <code>podman run</code>                          |
| <code>podman container ls</code>                                | <code>podman ps</code>                           |
| <code>podman image ls</code>                                    | <code>podman images</code>                       |
| <code>podman image pull</code>                                  | <code>podman pull</code>                         |
| <code>podman container rm</code> / <code>podman image rm</code> | <code>podman rm</code> / <code>podman rmi</code> |

Three diagnostic commands to know from now on:

```
podman version  # client version (and server, if there is one)
podman info     # engine state: rootless?, cgroups, network, storage, kernel
podman inspect  # every piece of metadata of an object, as JSON
```

#### PITFALL

With Docker, `docker version` shows two blocks, *Client* and *Server*; if the second is missing, the daemon is not running or you are not allowed to talk to it — the problem is never in your command. With Podman there is only one block: there is no server. The "Cannot connect to the Docker daemon" message you will find in every FAQ therefore does not exist... unless you use `podman --remote` or `podman machine`.

## 6. In the workplace

---

On a Spring Boot + Angular + PostgreSQL stack:

- The Spring Boot back end becomes **one image** containing a JRE and the JAR. The same image, down to the **digest**, goes to integration, staging and production: that is the end of "it works on my machine".
  - The Angular front end is *built* ( `ng build` ) and the static result is packed into an nginx image. Node does not survive into production — lab 05.
  - PostgreSQL is pulled from an official public image; you do not write it, you configure it.
  - These three images live in a **private registry**, pushed by CI. On the servers they run under Docker, Podman or Kubernetes: the image does not know who runs it, and that is the point.
- 

## Remember

---

- A container is a **process isolated** by *namespaces* and limited by *cgroups*, run by the **host kernel** — not a mini-VM. On Windows that kernel is WSL 2's.
- An **image** is an immutable template; a **container** is a live instance of it with a disposable writable layer.
- Docker = client + permanent root daemon; Podman = a program with no daemon, rootless by default. Same CLI, same images, same registries (OCI).
- Containers are **disposable**: any state not moved outside disappears when they are removed.
- Access to the Docker daemon = root access; rootless Podman grants only your rights.
- The CLI follows `podman <object> <action>`; the short forms ( `ps` , `run` , `images` ) are shortcuts.

## Vocabulary

---

**image**: immutable template, a stack of read-only layers. — **container**: a running instance of an image. — **layer**: a file-system fragment produced by one build step. — **registry**: server that stores and distributes images. — **repository**: all the versions of one image ( `postgres` ). — **tag**: label of one version ( `postgres:16-alpine` ). — **daemon**: permanent service ( `dockerd` ); absent in Podman. — **rootless**: mode in which the engine and the containers run under your own user. — **namespace**: kernel isolation of the view of a resource. — **cgroup**: consumption limit. — **runtime**: `runc` / `crun`, the component that actually creates the container. — **common**: Podman's small supervisor attached to each container. — **OCI**: the open standard for images and runtimes.